

AI-Powered Credit Risk Intelligence Platform

Lightweight Explainable Credit Scoring, Policy Rule Verification, and Conversational Analytics

Name: Divyanshi Raghuwanshi

GitHub Repository: https://github.com/DivyanshiRaghuwanshi/AI-powered_Credit_Risk_platform-

Live Deployment Link: <https://bunfsqlosfun8fnkjnxakv.streamlit.app/>

Project Objectives

- ▶ **Core Goal:** Build an end-to-end explainable credit scoring application using the Kaggle Home Credit Default Risk dataset.
- ▶ **Key Capabilities:**
 - ▶ **EDA & Ingestion:** Clean, engineer, and load application columns into PostgreSQL.
 - ▶ **ML Predictive Model:** Train robust classifiers to evaluate default probabilities.
 - ▶ **Explainable AI (XAI):** Calculate local SHAP values and translate numerical feature weights into plain English explanations.
 - ▶ **Conversational Agent:** Build an NL-to-SQL chatbot with an auto-correct/self-repair execution loop.
 - ▶ **Credit Policy Rules:** Auto-extract logic rules and validate them using proportion Z-tests before storing them in a rule registry.

System Architecture & Tech Stack

► Three-Tier Architecture:

- **UI Layer:** Streamlit web interface for business analysts and underwriters.
- **API Layer:** FastAPI backend acting as the routing agent and LLM gateway.
- **Database:** PostgreSQL storing processed records, schema descriptions, and engineered tables.

► Technology Stack:

- Python 3.11, Uvicorn, FastAPI, Streamlit, Pandas, NumPy.
- Scikit-Learn (Modeling), SHAP (Explainability).
- LangChain, ChatOpenAI (supporting Groq Llama 3, Gemini 1.5, or OpenAI GPT-4o-mini).

Data Processing & EDA Insights

- ▶ **Ingested Volume:** 356,255 rows and 92 columns loaded from home_credit_future_dpd_labels.csv.
- ▶ **Preprocessing Pipeline:**
 - ▶ Features selected based on credit relevance (external ratings, debt-to-income, credit utilization).
 - ▶ Missing value imputation using median statistics to handle columns with high null values.
 - ▶ Standard scaling applied to variables before feeding into models.
- ▶ **Key Observations:**
 - ▶ Extreme target label imbalance: ~91.9% active/repaid accounts vs. ~8.1% default accounts.
 - ▶ Strong negative correlation between external rating variables (EXT_SOURCE_2, EXT_SOURCE_3) and default risk.

Predictive ML Modeling

► Classifier Model Selection:

- **Random Forest Classifier:** Primary model selected for its resistance to overfitting, handling of non-linear interactions, and compatibility with SHAP explainer objects.
- **Decision Tree Classifier:** Secondary model used as the structural engine to extract cutoff limits.

► Class Imbalance Strategy:

- Integrated class cost penalization directly into modeling (class_weight="balanced" / balanced_subsample).

► Validation Performance:

- Evaluated on Stratified Splits (preserving default label ratios).
- Achieved validation **ROC-AUC of 0.71 to 0.75**, ensuring strong discriminative capability.

Explainable AI (SHAP + LLM)

- ▶ **Underwriting Transparency:**

- ▶ Individual client scoring outputs a default probability % and a Risk Band (Low, Medium, High).

- ▶ **SHAP TreeExplainer:**

- ▶ Computes mathematical local feature importance weights for every prediction.

- ▶ **Narrative Synthesis:**

- ▶ Compiles the top positive drivers (factors raising risk) and negative drivers (factors lowering risk).
- ▶ Feeds these values to the LLM (Groq/Gemini/OpenAI) to generate a business-friendly justification (narrative description) and recommended actions for the Relationship Manager (RM).

Conversational Talk-to-Data Chatbot

- ▶ **Objective:** Allow banking analysts to query data using natural language (NL-to-SQL).
- ▶ **Execution Flow:**
 - ▶ User asks a question (e.g. *"What is the average credit score of defaulting customers?"*).
 - ▶ Prompt constructs system context using Postgres column description metadata.
 - ▶ LLM translates the query into syntax-accurate SQL.
- ▶ **Self-Repair Execution Loop:**
 - ▶ If the database encounters an execution or syntax error, the backend catches the traceback, sends it back to the LLM with the error logs, and automatically regenerates and executes the repaired SQL query.

Statistical Credit Policy Rules

► ML to Policy Rules:

- Bridge the gap between black-box predictions and human-readable credit policy criteria.

► Cutoff Extraction:

- Extracts decision tree splits to form explicit policy rules (e.g. `EXT_SOURCE_2 < 0.35 AND AMT_INCOME_TOTAL < 120000`).

► Mathematical Validation:

- Calculates segment default rate vs. base default rate (**Lift**).
- Executes a **two-sample proportion Z-test** against the complement.
- Only rules that reject the null hypothesis (H_0 : no difference in default rates) with statistical significance (p -value < 0.05) are saved to the persistent `approved_rules.json` registry.

Docker Containerization & Fallback Notice

► Docker Packaging:

- A Docker environment is fully configured with a root Dockerfile and docker-compose.yml.
- Services are containerized into three decoupled tasks: postgres database, api FastAPI service, and ui Streamlit dashboard.

► Localhost Verification:

- During local environment development, WSL/host network socket pipe timeouts occurred on the Docker daemon.
- Consequently, the platform was verified and run using **Standard Localhost Execution (Option B)** (FastAPI and Streamlit run in a local .venv environment) to ensure complete runtime correctness. Both running options are fully configured.

Conclusion & Outputs

► Summary of Achievements:

- Successfully delivered data preprocessing, database loading, ML classification, and SHAP explainability.
- Implemented an advanced SQL agent with self-repair capabilities and a statistically sound policy validation registry.

► Screenshots Attached:



